

SEC101: PROBLEM SOLVING TECHNIQUES

Course Code	SEC101
Credits	5
L:T:P	3L:0T:4P
Total Lectures / Practical Hours	45 theory lectures + 60 practical hours
Academic Level	Foundation / Introductory Skill Course
Semester	I
Prerequisite	This is an introductory programming course and hence no prerequisites.

1. INTRODUCTION

Course Rationale

This course introduces students to computational problem-solving, algorithmic thinking, pseudocode, flowcharts, structured programming and C programming. It trains learners to move from problem requirements to specifications, algorithms, code, testing and debugging.

Course Goals

- To help students understand computational problems and their classifications.
- To train students in algorithm design using sequence, selection and repetition.
- To develop the ability to translate algorithms into C programs.
- To introduce testing, debugging, modular programming, arrays, recursion and basic string handling.

Course Objectives:

- CO1: Understand basic terminology of computers, problem solving, programming languages and their evolution.
- CO2: Create specification from problem requirements by asking questions to disambiguate the requirement statement.
- CO3: Design the solution from specification of a problem and write pseudo code of the algorithm using structured programming constructs: sequence, selection and repetition statements.
- CO4: Translate an algorithm into a C computer program.
- CO5: Test and analyse programs using debugging tools.

Learning Outcomes:

- LO1: Analyse a problem and identify its inputs, outputs, constraints and special cases.
- LO2: Write pseudocode and flowcharts for structured problems.
- LO3: Implement basic C programs using input/output, operators, conditionals and loops.
- LO4: Solve problems involving numbers, sequences, arrays, matrices, strings and recursion.
- LO5: Debug, test and refine programs using incremental compilation and good coding practices.

2. COURSE STRUCTURE (45 THEORY LECTURES + 60 PRACTICAL HOURS)

(Theory lecture plan has been added; the original laboratory problem list has been retained below.)

UNIT I: Problems, Algorithms and Problem-Solving Process (10 Lectures)

Topics:

- Problems and problem instances, generalization and special cases, types of computational problems and classification of problems.
- Analysis of problems, solution approaches, algorithm development, analysis of algorithm, efficiency and correctness.
- Role of data structures in problem solving.
- Problem-solving steps: understand the problem, plan, execute and review.

- Breaking the problem into subproblems, input/output specification, input validation, pre and post conditions.

Practical / Activity:

- Requirement clarification and input-output specification exercises for simple computational problems.

Lecture Plan:

- L1-L2: Problems, problem instances, generalization and special cases.
- L3-L4: Types and classification of computational problems.
- L5-L6: Problem analysis, solution approaches and algorithm development.
- L7: Efficiency, correctness and role of data structures.
- L8-L10: Problem-solving steps, subproblems, input/output specifications and validation.

UNIT II: Structured Programming, Data Representation and C Basics (15 Lectures)

Topics:

- Structured programming concepts: sequence, selection and repetition statements; control structure stacking and nesting.
- Different kinds of repetitions: entry controlled, exit controlled, counter controlled, definite, indefinite and sentinel-controlled repetitions.
- Pseudocode and flowcharts; definition and characteristics of algorithms; standard algorithm format.
- Problems involving iteration and nesting: patterns, arithmetic and geometric progression, Fibonacci and other sequences, approximate values of π , $\sin(x)$, $\cos(x)$ using Taylor series.
- Representation of integers: signed magnitude, 1's complement and 2's complement; real numbers using IEEE 754; character representation through ASCII and UNICODE.
- C language: programming languages, generations, typed vs typeless programming languages, history of C, empty C program, `scanf()`, `printf()`, assignment, arithmetic, relational and logical operators, if, if-else, for, while, do-while, data types, translation from pseudocode to C, incremental compilation and testing, good coding practices.

Practical / Activity:

- Programming exercises on input/output, conditionals, loops, patterns, sequences and Taylor series approximations.

Lecture Plan:

- L11-L12: Structured programming constructs: sequence, selection and repetition.
- L13-L14: Types of loops and nested control structures.
- L15-L16: Pseudocode, flowcharts and algorithm format.
- L17-L19: Iterative problems, patterns and sequences.
- L20-L21: Taylor series approximations and numerical representation.
- L22-L23: Integer, real and character representation.
- L24-L25: C language introduction, operators, data types and basic statements.

UNIT III: Number Problems and C Control Statements (10 Lectures)

Topics:

- Problems on numbers: extracting digits of a number left to right and right to left, palindrome, prime number, prime factors, amicable number, perfect number, Armstrong number, factorial and number base conversion.
- Statistics such as maximum, minimum, sum and average on a sequence of numbers read using sentinel-controlled repetition using only a few variables.
- C language: else-if ladder, switch case, increment/decrement operators, break and continue statements.

Practical / Activity:

- Programs on number classification, grading, base conversion, sentinel-controlled input and BMI classification.

Lecture Plan:

- L26-L27: Digit extraction, palindrome and number analysis.

- L28-L29: Prime, prime factors, perfect, amicable and Armstrong numbers.
- L30-L31: Factorial and base conversion.
- L32-L33: Sentinel-controlled statistics problems.
- L34-L35: Else-if ladder, switch case, increment/decrement, break and continue.

UNIT IV: Modular Programming, Arrays, Strings and Recursion (10 Lectures)

Topics:

- Modular programming, top-down and bottom-up approaches to problem solving.
- Recursion.
- Problems on arrays: reading and writing array elements, maximum, minimum, sum, average, median and mode.
- Sequential and binary search; any one sorting algorithm; matrix operations.
- C language: function definition and declaration, role of return statement, one-dimensional and two-dimensional arrays, string functions, other operators, operator precedence and associativity, debugging.

Practical / Activity:

- Programs on arrays, matrices, strings, functions, recursion and debugging.

Lecture Plan:

- L36: Modular programming and top-down / bottom-up approaches.
- L37-L38: Functions and return statements in C.
- L39-L40: Arrays and basic array problems.
- L41: Searching and sorting.
- L42: Matrix operations.
- L43: Strings and string functions.
- L44: Recursion.
- L45: Debugging and integrated review.

PRACTICAL / LAB COMPONENT: PROBLEM SOLVING TECHNIQUES

- UNIT-II: Convert degrees Celsius to Fahrenheit and vice versa.
- Display three input numbers in sorted non-decreasing order.
- Given $n \geq 0$, display number, square and cube of numbers from 1 to n in tabular format.
- Given a positive integer n, display odd numbers in the range [1, n].
- Display mathematical tables; generalize to first n tables up to m rows.
- Display specified symbol and number patterns for n rows.
- Display arithmetic / geometric progression terms from first term, difference / multiplier and number of terms.
- Display Fibonacci and Tribonacci sequences.
- Check whether two positive integers are consecutive Fibonacci numbers.
- Compute approximate values of π , e^x , $\sin(x)$ and $\cos(x)$ using Taylor series.
- UNIT-III: Extract digits of an integer number left to right and right to left.
- Form a number from a sequence of digits using sentinel-controlled repetition.
- Check palindrome, prime, amicable, perfect and Armstrong numbers.
- Compute grades using else-if ladder and switch case.
- Compute sum, maximum, minimum and average of a sentinel-controlled sequence.
- Convert a positive integer from one base to another after input validation.
- Display a number in text form.
- Create grade-frequency bar chart using sentinel-controlled repetition.
- Compute BMI and classify underweight, normal, overweight or obese.
- UNIT-IV: Modularized program for circular prime.
- Modularized program to compute maximum of 8 numbers.

- Array programs for mean, range, mode and median.
- Implement string length and string reversal functions.
- Matrix operations: addition, subtraction and transpose.
- Recursive programs for digit count, factorial, digit display, x^y and reverse printing of input sequence.

3. PEDAGOGICAL APPROACH

- Hands-on problem-solving before syntax-heavy programming.
- Algorithm-first teaching through pseudocode, flowcharts and dry runs.
- Incremental compilation, testing and debugging in the programming lab.
- Frequent movement from everyday problems to computational formulations.
- Use of modular programming and code review to build good programming habits.

4. SUGGESTED TEXTUAL / ACTIVITY COMPONENT

- Short exercises after every concept cluster.
- Mini assignments and classroom demonstrations.
- Peer explanation of selected problems or concepts.
- Use of diagrams, flowcharts, code fragments, truth tables or hardware layouts wherever relevant.

5. SKILL APPLICATIONS OF THE PAPER

- Analytical thinking and formal representation.
- Computational problem-solving and structured reasoning.
- Technical communication through diagrams, algorithms, code or lab records.
- Readiness for advanced BCA courses and applied computing tasks.

6. MAIN TEXTS / REFERENCE BOOKS

- Venkatesh, Nagaraju Y. Practical C Programming for Problem Solving. Khanna Book Publishing Company, 2024.
- AICTE's Programming for Problem Solving (with Lab Manual). Khanna Book Publishing Company, 2024.
- Deitel, Harvey, and Paul Deitel. C How to Program. 9th ed., Pearson India, 2015.
- Dromey, R. G. How to Solve It by Computer.
- Kernighan, Brian W., and Dennis Ritchie. The C Programming Language. 2nd ed., Pearson, 2015.
- Hanly, Jeri, and Elliot Koffman. Problem Solving and Program Design in C. 8th ed., Pearson, 2015.

7. ASSESSMENT PATTERN

The following assessment pattern is suggested for model-aligned presentation and may be finalized by the Board of Studies according to institutional regulations.

Component	Suggested Weightage	Suggested Evidence	Remarks
Internal Assessment	30%	Class tests, quizzes, assignments, presentation, lab record where applicable	Added for model consistency
End-Semester Theory Examination	50-70%	Written examination with conceptual, analytical and problem-solving questions	Weightage to be finalized by the institution
Practical / Lab Assessment	20-30%	Lab programs, viva, debugging, record book, demonstration	Relevant for laboratory-based BCA courses

8. FINAL PEDAGOGICAL INSIGHT

Problem Solving Techniques should be taught as a bridge between conceptual clarity and computing practice. The course should move students from definitions to representation, and then from representation to problem-solving.